📱 Mobile App Development & DevOps Complete Reference Guide

Part 1: Core Concepts & Tools

App ID (Application Identifier)

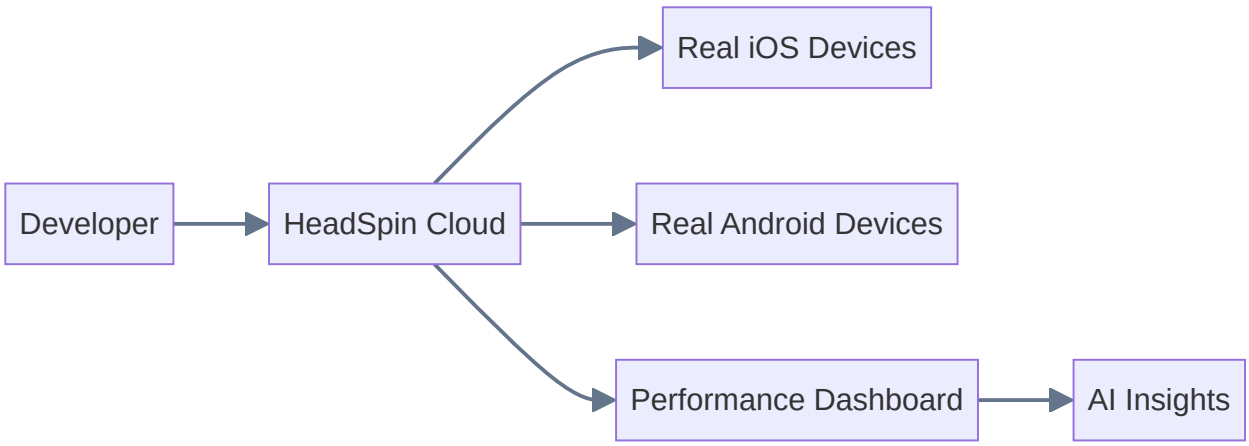
A unique identifier for a mobile application used by app stores and OS platforms.

Platform	Format	Example
iOS	Reverse-domain Bundle ID	`com.company.myapp`
Android	Application ID (package name)	`com.company.myapp`

- Registered in **Apple Developer Portal** (iOS) or **Google Play Console** (Android)
- Must be globally unique across the entire store
- Cannot be changed after publishing

HeadSpin

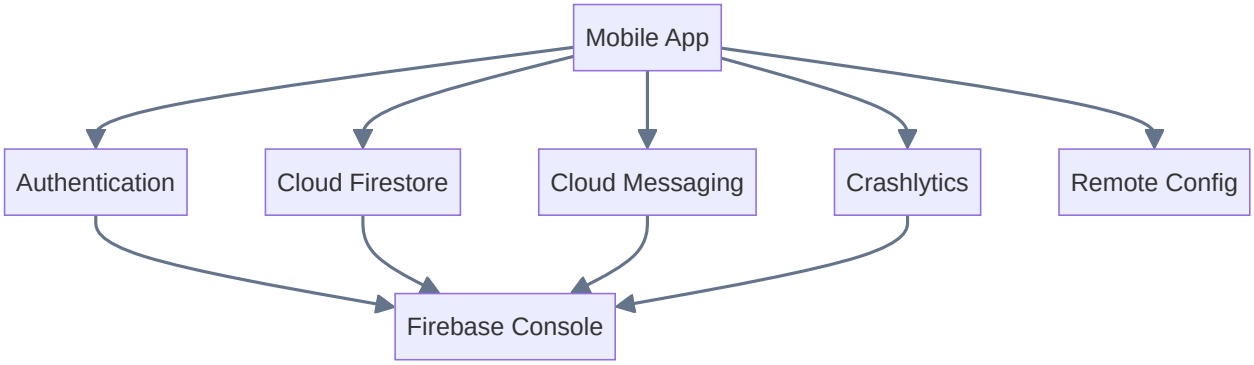
A **mobile testing and performance platform** that provides real device cloud testing, AI-driven performance analytics, and CI/CD integration.



Firebase

Google's **Backend-as-a-Service (BaaS)** platform for mobile and web apps.

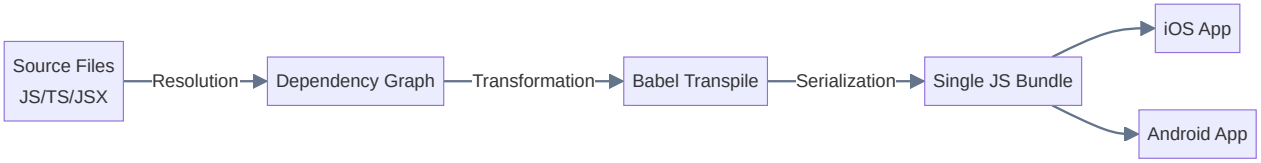
Service	Purpose
Authentication	Email, Google, Apple, Phone sign-in
Cloud Firestore	NoSQL real-time database
Cloud Messaging (FCM)	Push notifications
Crashlytics	Crash reporting & analytics
Remote Config	Change app behavior without deploy
App Distribution	Beta testing distribution
Analytics	User behavior tracking



Metro Bundler

The **JavaScript bundler** for React Native — equivalent of Webpack but optimized for RN.

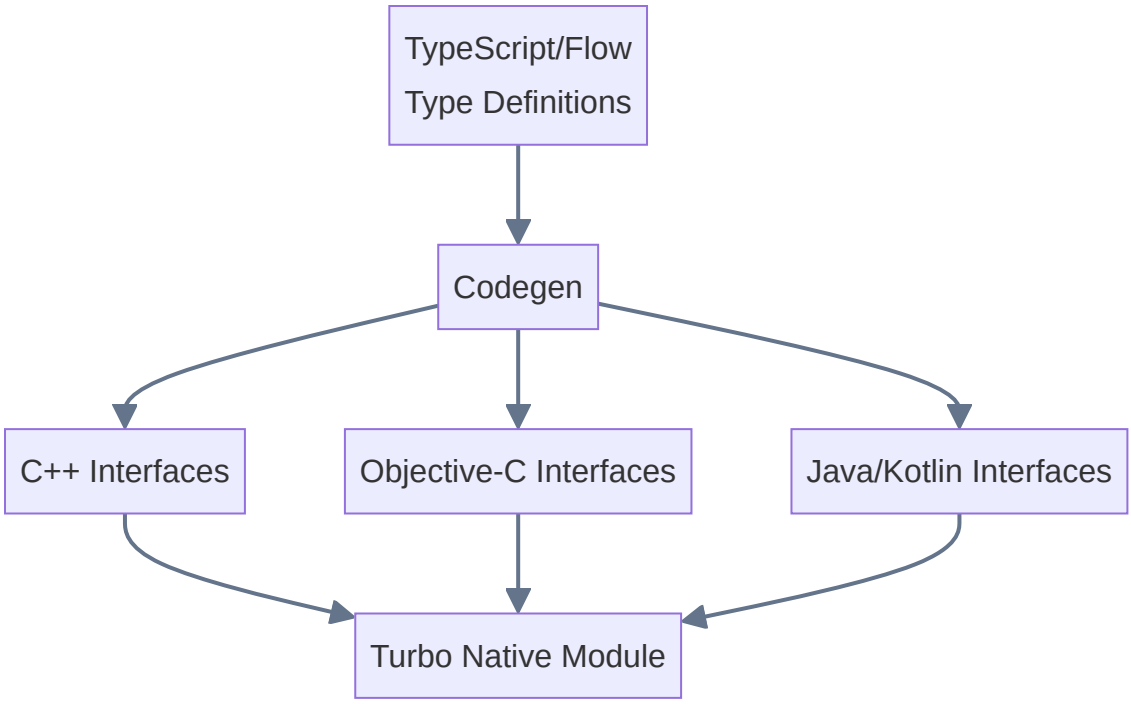
1. **Resolves** all `import` / `require` into a dependency graph
2. **Transforms** code using Babel (JSX → JS, TS → JS)
3. **Bundles** everything into a single JS file
4. **Serves** via local dev server with Hot Module Replacement



Key config: `metro.config.js`

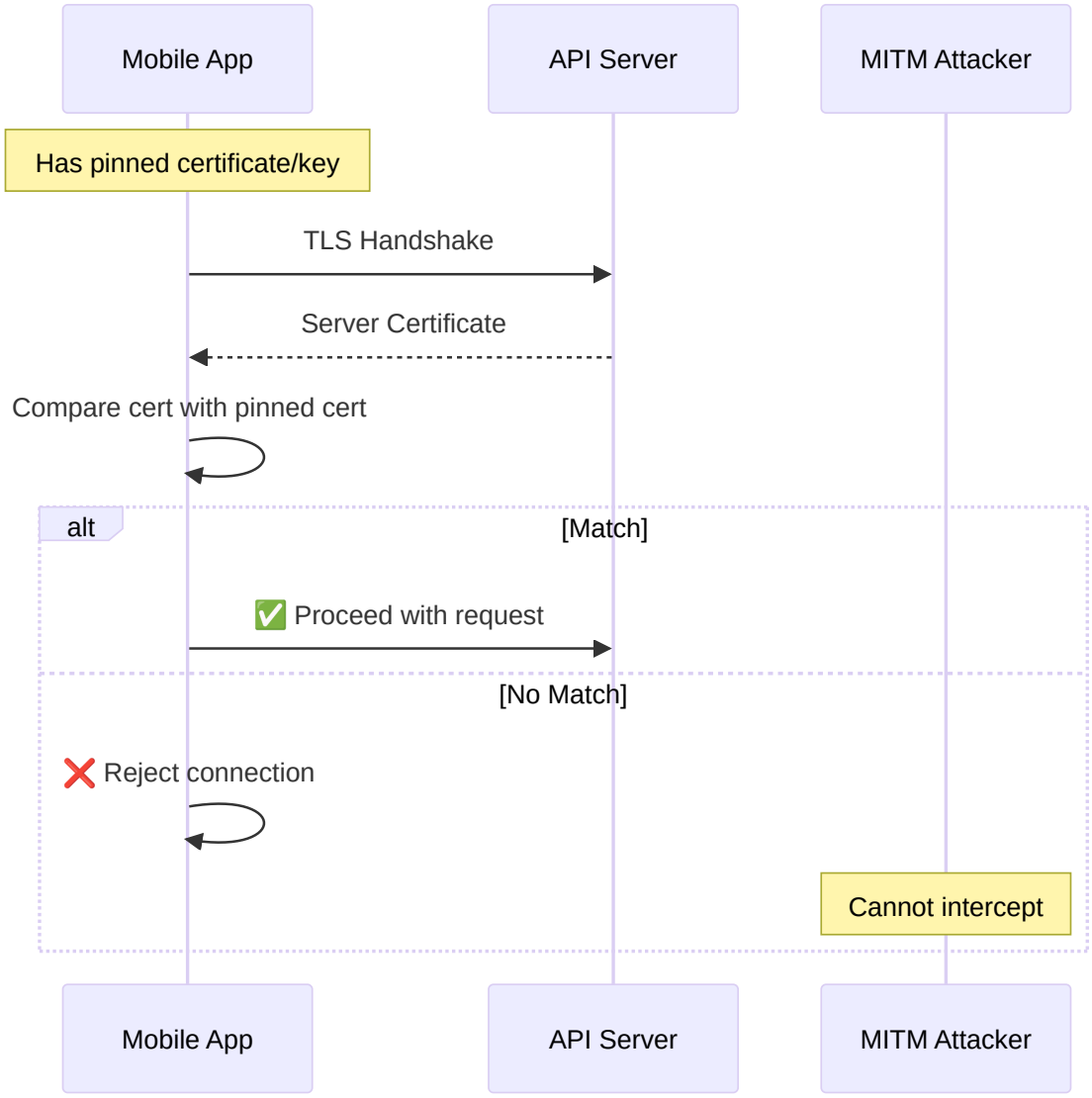
Codegen (React Native)

A **code generation tool** for React Native's New Architecture (Turbo Modules & Fabric). Generates native interface code from TypeScript type definitions for type-safe JS↔Native communication.



SSL Pinning

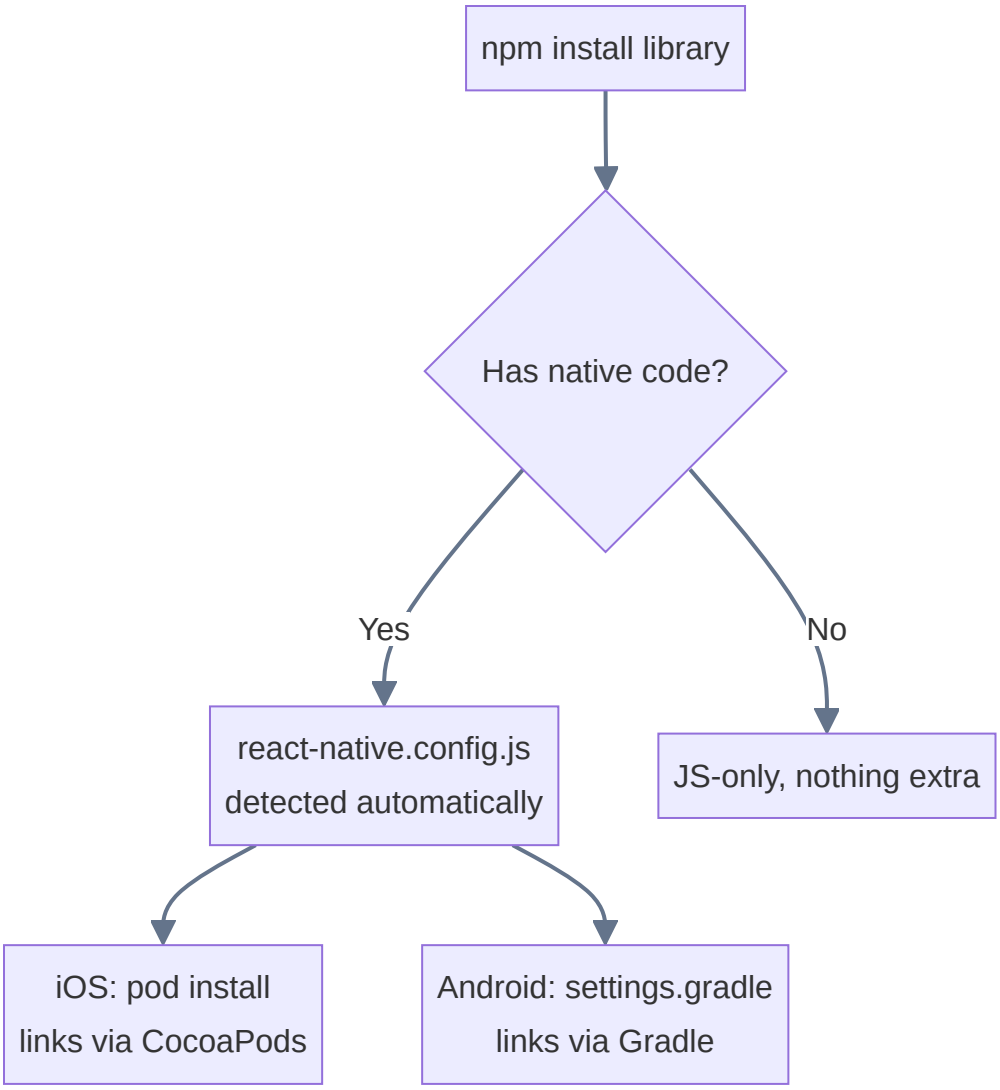
A security technique that **hardcodes** the server's SSL certificate or public key inside the app, preventing MITM attacks.



Type	Pins	Flexibility
Certificate Pinning	Entire certificate	Least flexible, must update app on cert renewal
Public Key Pinning	Only the public key	More flexible, survives cert renewals
Hash Pinning	SHA-256 hash of key	Most common approach

Auto-Linking (React Native 0.60+)

Automatically links native dependencies when you install an npm package that contains native code. No more `react-native link`.



---

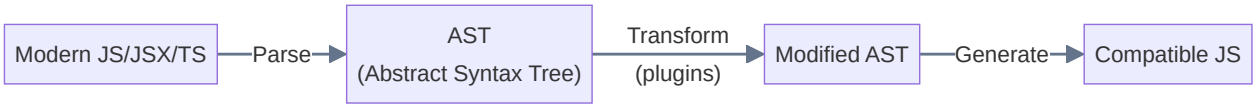
npm vs pnpm vs yarn

All three are **JavaScript package managers** that install dependencies from the npm registry.

Feature	npm	yarn	pnpm
Created by	npm Inc.	Meta	Zoltan Kochan
Lock file	`package-lock.json`	`yarn.lock`	`pnpm-lock.yaml`
Speed	Moderate	Fast (parallel)	Fastest (hardlinks)
Disk usage	Duplicates	Duplicates	**Shared store**
Monorepo	Workspaces (basic)	Workspaces (mature)	Workspaces (excellent)
Phantom deps	Allowed	Allowed	**Prevented**
Default in Node	✅	❌	❌

Babel

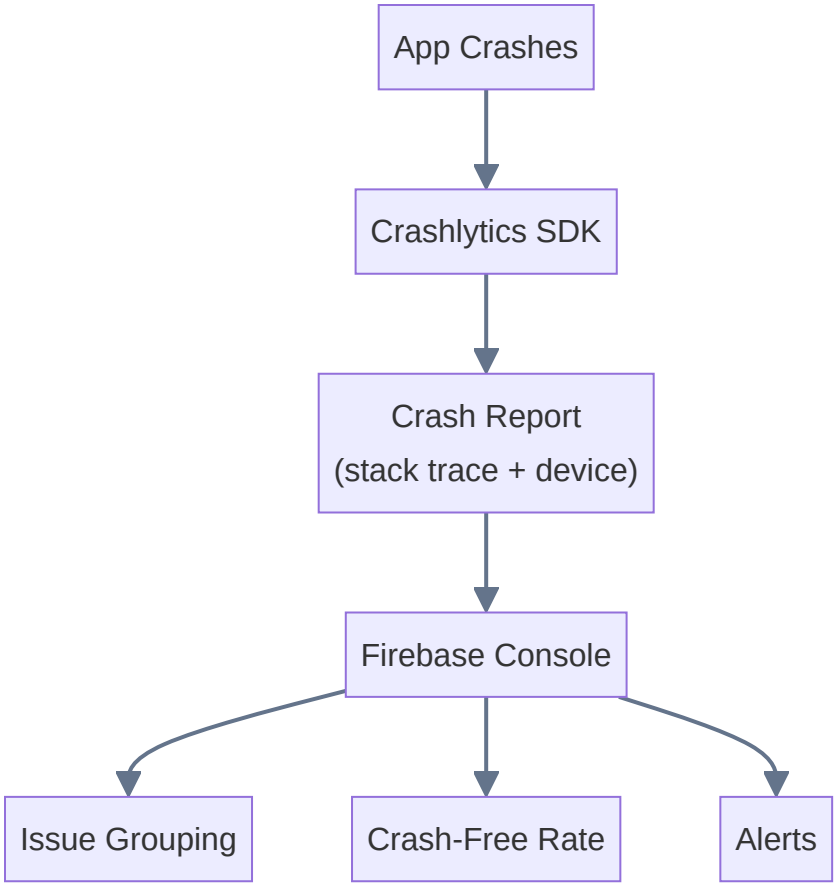
A **JavaScript compiler/transpiler** that converts modern JS (ES2015+, JSX, TypeScript) into backwards-compatible code.



Key config: `babel.config.js`

Crashlytics (Firebase)

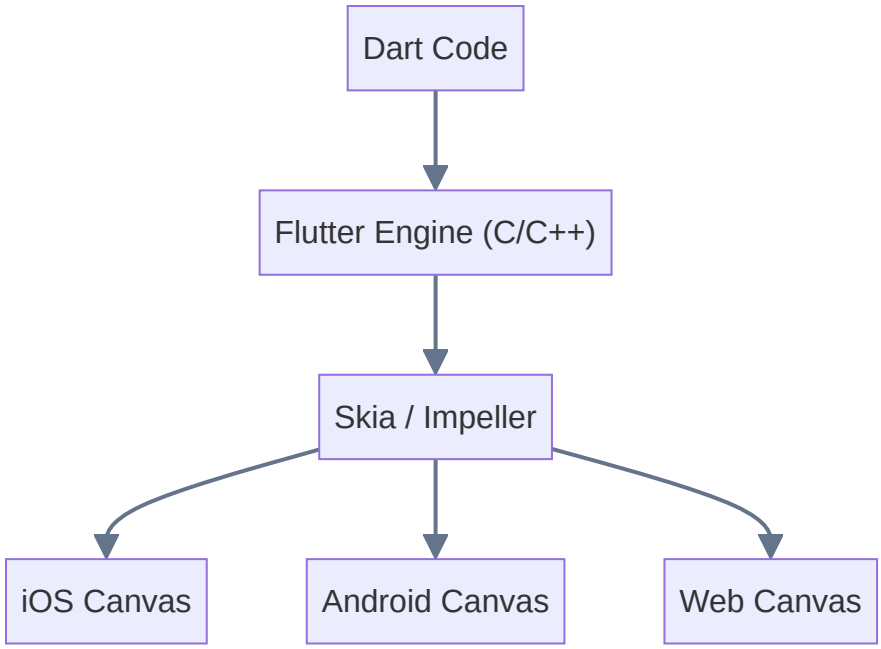
Real-time **crash reporting tool**. Captures crash logs, stack traces, device info. Groups crashes by root cause.



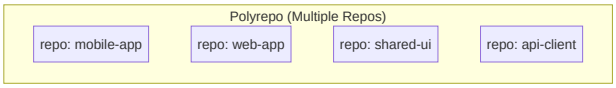
Flutter

Google's **cross-platform UI toolkit** using **Dart**. Renders via its own engine (Skia/Impeller), not platform widgets.

Feature	Flutter	React Native
Language	Dart	JavaScript/TypeScript
Rendering	Own engine (Skia/Impeller)	Native platform widgets
Hot Reload	✅	✅
Architecture	Widget tree	Component tree + JSI



Monorepo vs Polyrepo



Aspect	Monorepo	Polyrepo
Structure	All in one repo	Each project separate
Code sharing	Easy (direct imports)	Requires publishing packages
CI/CD	Complex (affected-only)	Simple (per-repo)
Tooling	Nx, Turborepo, Lerna	Standard Git
Examples	Google, Meta	Most startups

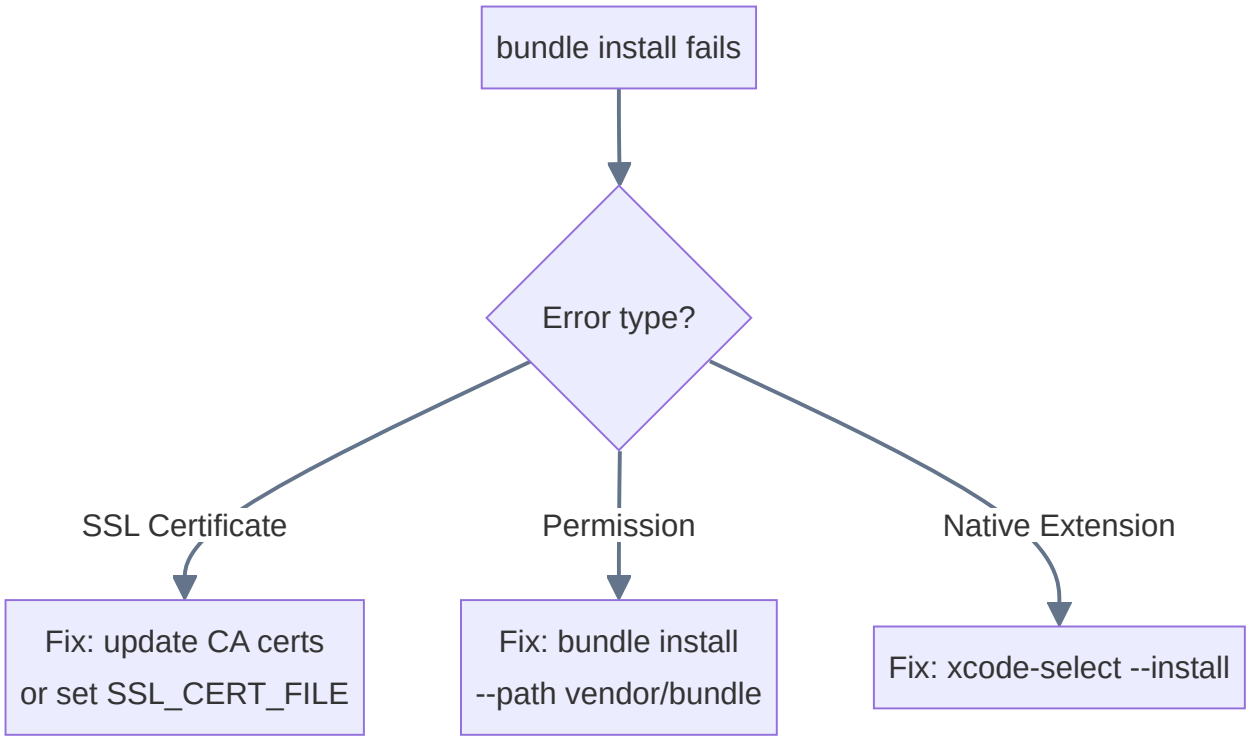
Part 2: Project Configuration & Files

1. package.json — dependencies vs devDependencies

```
{
  "dependencies": {       // ← ships with app
    "react": "^18.2.0",
    "axios": "^1.4.0"
  },
  "devDependencies": {    // ← build & test only
    "jest": "^29.0.0",
    "eslint": "^8.0.0",
    "typescript": "^5.0.0"
  }
}
```

	dependencies	devDependencies
Purpose	Required at **runtime**	Required during **dev/build**
In production	✅ Yes	❌ No
Examples	React, Axios, Express	Jest, ESLint, TypeScript
Install flag	`npm install pkg`	`npm install pkg --save-dev`

2. Local Setup — Certificate & Bundle Install Issues



Problem	Fix
SSL cert verification failed	`export SSL_CERT_FILE=/path/to/cacert.pem`
Corporate proxy intercepting	Add corporate CA to trust store
Expired system certs	`brew install openssl` + update Ruby SSL config

---

3. .plist File (Property List)

An XML/binary key-value config file for iOS/macOS. **Info.plist** = app metadata read at launch.

```
<dict>
    <key>CFBundleIdentifier</key>
    <string>com.company.myapp</string>
    <key>CFBundleVersion</key>
    <string>42</string>
    <key>NSCameraUsageDescription</key>
    <string>We need camera for scanning</string>
</dict>
```

Key	Purpose
`CFBundleIdentifier`	Bundle ID (App ID)
`CFBundleVersion`	Build number
`CFBundleShortVersionString`	User-facing version (e.g. "2.1.0")
`NSCameraUsageDescription`	Camera permission prompt text

---

4. .m Files (Objective-C Implementation)

Contains the **implementation** of classes and methods. Equivalent to `.cpp` in C++.

```
// MyModule.m
#import "MyModule.h"
@implementation MyModule
- (void)doSomething {
    NSLog(@"Doing something!");
}
@end
```

---

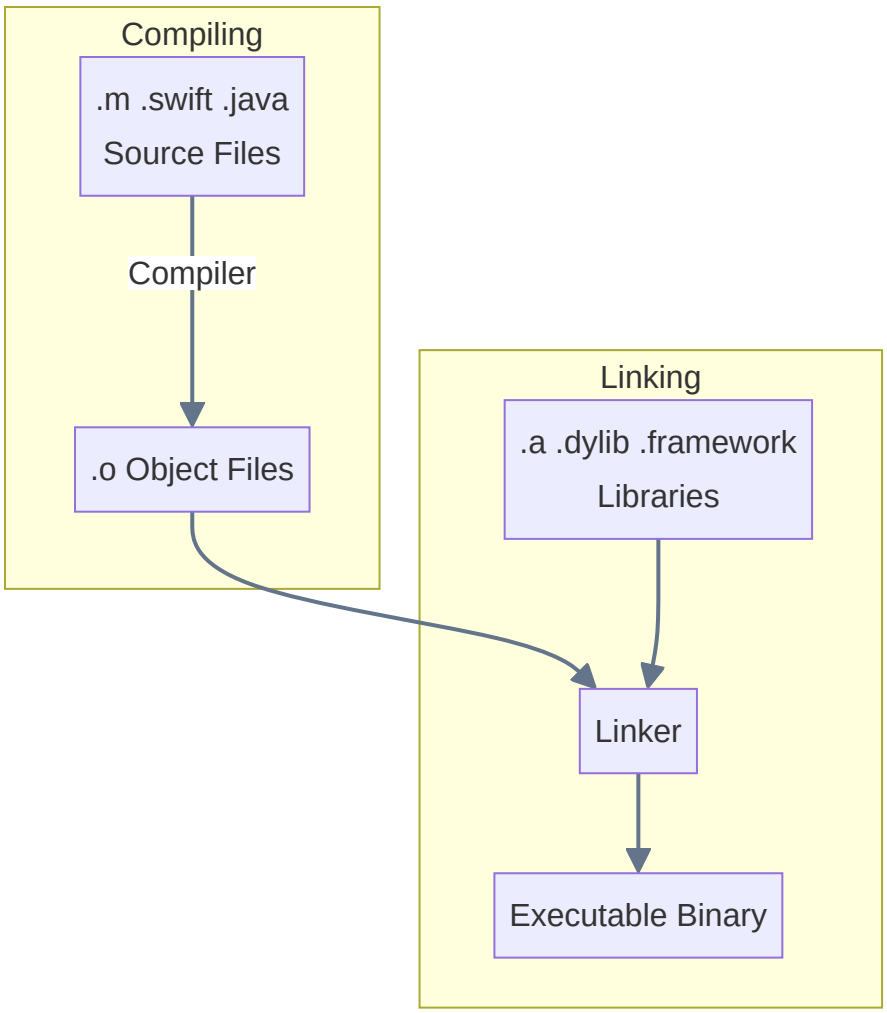
5. .h Files (Header/Declaration Files)

Contains **declarations** — class interfaces, method signatures, constants. Shared via `#import`.

```
// MyModule.h
#import <Foundation/Foundation.h>
@interface MyModule : NSObject
- (void)doSomething;
@end
```

---

6. Compiling and Linking



Phase	What Happens
Compiling	Each source file → machine code (object file). Syntax checked, types verified.
Linking	All object files + libraries → single executable. Resolves cross-file references.

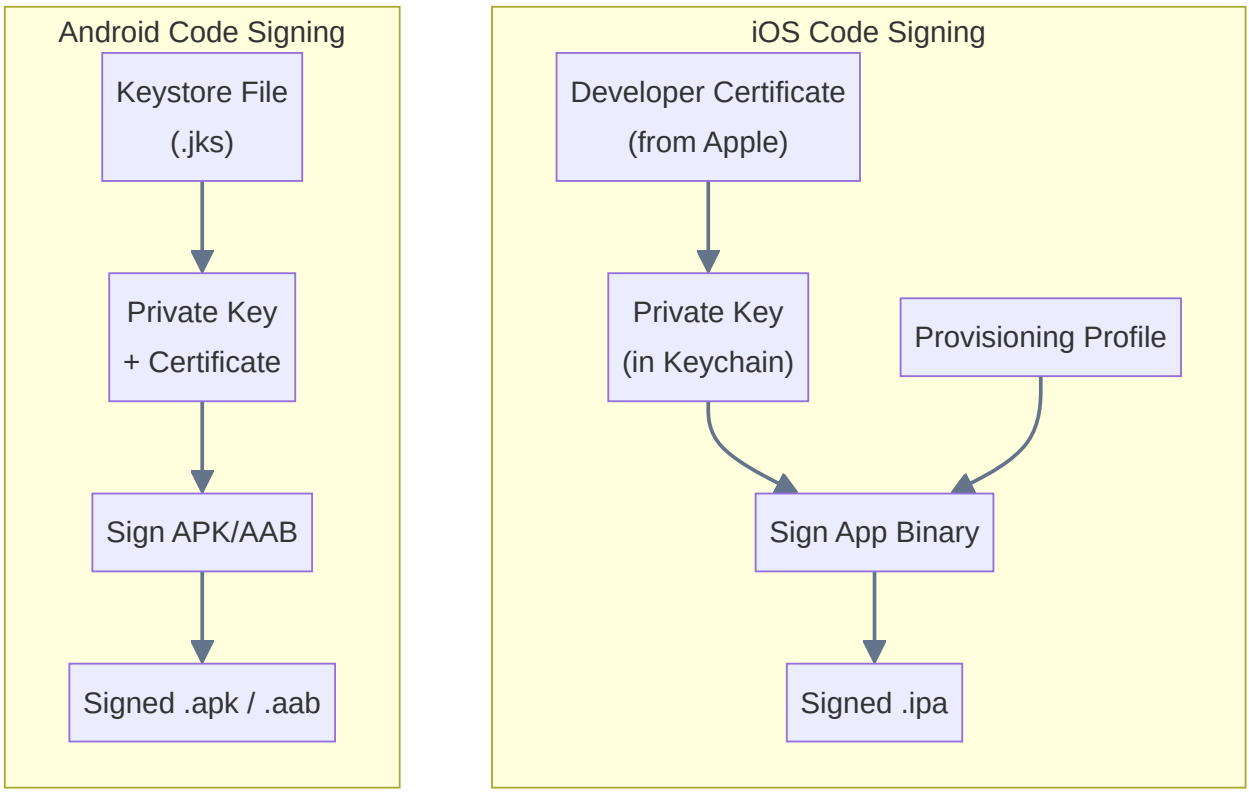
Static linking: library code copied into binary (larger, self-contained)

Dynamic linking: library loaded at runtime (smaller binary, shared libs needed)

---

7. Code Signing (iOS & Android)

Cryptographically guarantees the app is from a known developer and hasn't been tampered with.



	iOS	Android
Authority	Apple Developer Program	Self-signed or Play App Signing
Key storage	macOS Keychain	Java Keystore (.jks)
Required for	Any installation	Release builds + Play Store

---

8. Code Sign Identity

Combination of a **certificate** (public key from Apple) + **private key** (in Keychain).

Identity	Use Case
`Apple Development`	Development builds (on-device testing)
`Apple Distribution`	App Store & Ad Hoc distribution

Verify: `security find-identity -v -p codesigning`

---

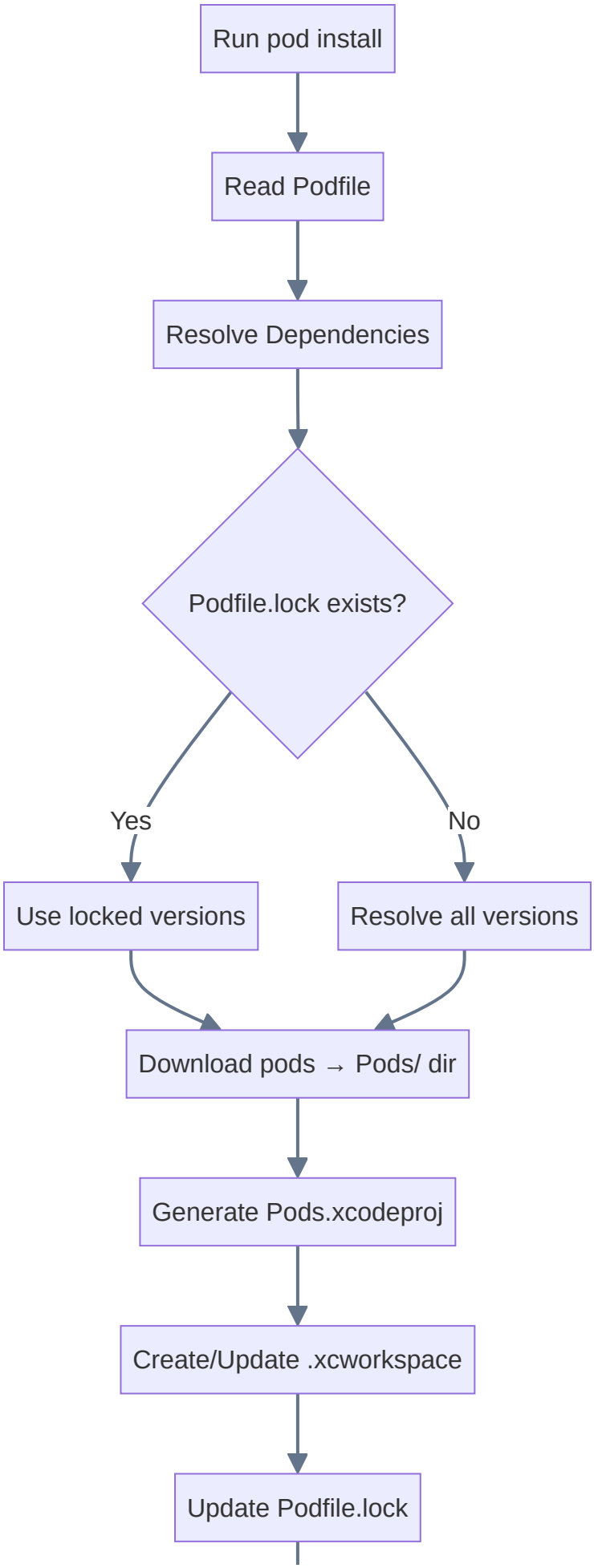
9. Gradle Commands

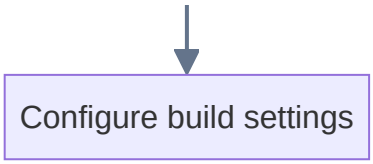
```
gradle --version                # Check Gradle version
./gradlew build                 # Build project (use wrapper!)
./gradlew assembleDebug         # Build debug APK
./gradlew assembleRelease       # Build release APK
./gradlew clean                 # Clean build outputs
./gradlew tasks                 # List all tasks
./gradlew dependencies          # Show dependency tree
```

> 💡 **Tip:** Always use `./gradlew` (Gradle Wrapper) over `gradle`. The wrapper ensures the correct version specified by the project.

---

10. What Happens During `pod install`?


Run pod install
Read Podfile
Resolve Dependencies
Podfile.lock exists?
Yes
No
Use locked versions
Resolve all versions
Download pods → Pods/ dir
Generate Pods.xcodeproj
Create/Update .xcworkspace
Update Podfile.lock



> ⚠️ **Important:** After `pod install`, always open `.xcworkspace` — not `.xcodeproj`.

---

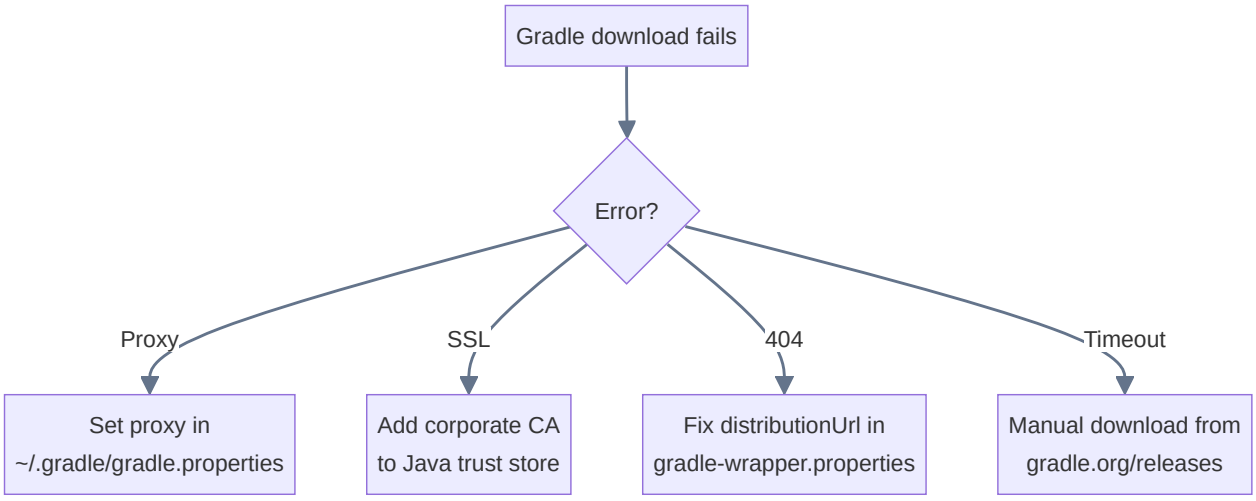
11. `brew install parallel`

GNU Parallel — shell tool for executing jobs in parallel.

```
brew install parallel
brew update && brew install parallel   # if first attempt fails
brew doctor                            # check brew health
```

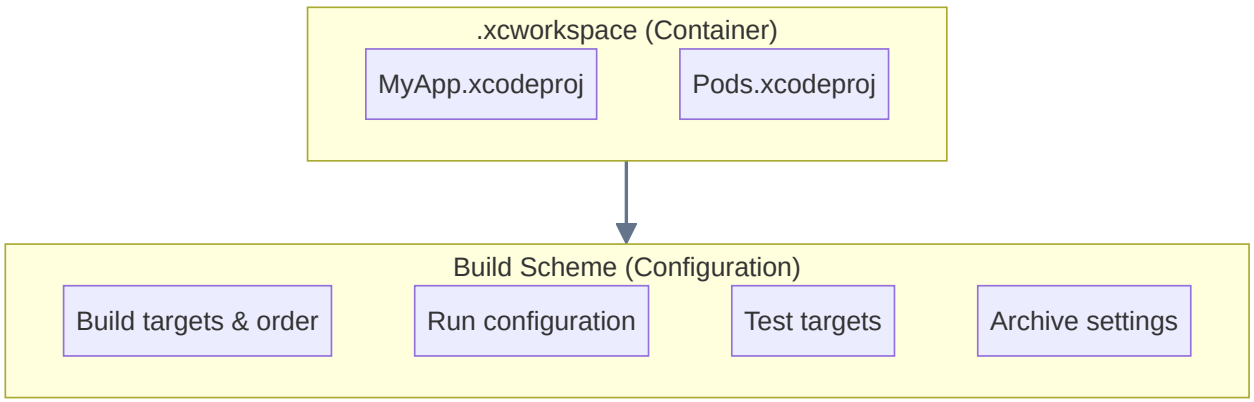
---

12. Gradle Download Issues



---

13. Scheme vs Workspace



Concept	What It Is
Workspace	Container grouping multiple Xcode projects
Scheme	Named config: what to build, how, with what build config

14. Scheme File

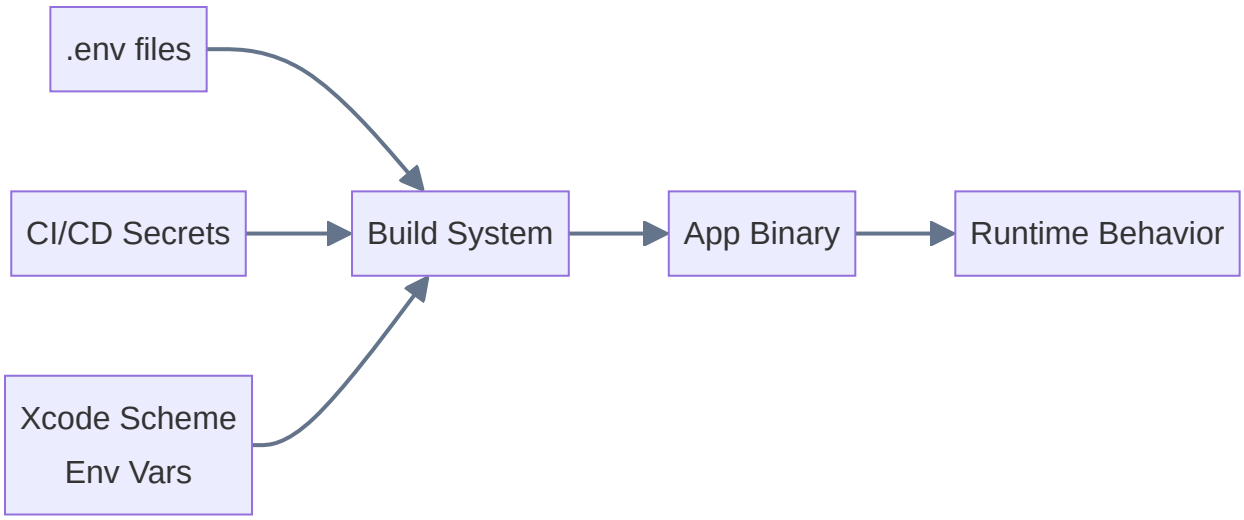
Stored at: `*.xcodeproj/xcshareddata/xcschemes/MyScheme.xcscheme` (XML)

Section	Controls
BuildAction	Which targets to build and order
LaunchAction	Run config (debug/release), env vars
TestAction	Which test targets to execute
ArchiveAction	Settings for distributable archives
ProfileAction	Instruments profiling configuration

15. Build Configurations

Config	Optimization	Debug Symbols	Use Case
Debug	None (`-O0`)	Included	Development
Release	Optimized (`-Os`)	Stripped	Production
Staging (custom)	Optimized	Included	Staging API

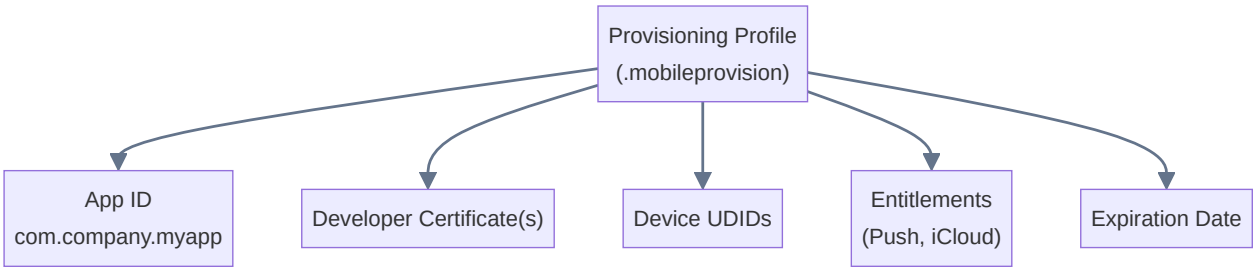
16. Environment Variables



Platform	Method
React Native	`react-native-config` reads `.env`
iOS Native	Xcode Scheme env vars or `.xcconfig`
Android	`buildConfigField` in `build.gradle`

17. Provisioning Profile

Apple-issued file binding together App ID, certificates, device UDIDs, and entitlements.



Type	Devices	Distribution
Development	Registered devices	Xcode install
Ad Hoc	Up to 100 devices	Direct distribution
App Store	Any device	App Store / TestFlight
Enterprise	Any in org	In-house

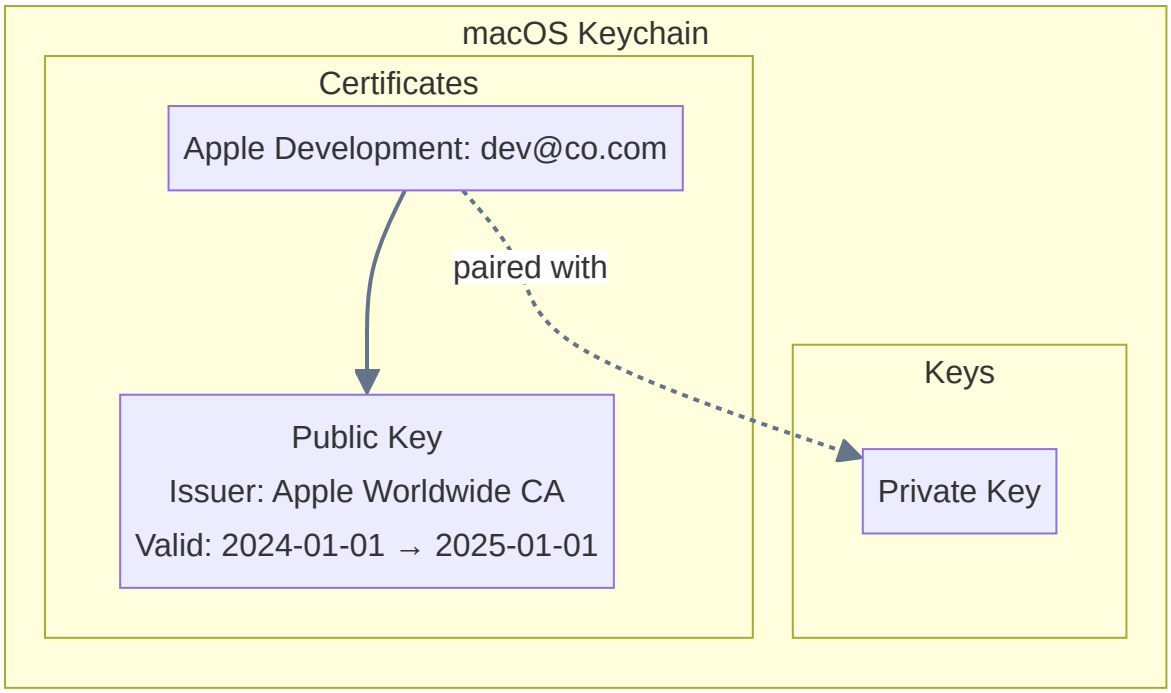
18. Developer Certificate

Digital certificate from Apple identifying you as a trusted developer.

Certificate	Purpose
Apple Development	Signing for development/testing
Apple Distribution	Signing for App Store / Ad Hoc

Get via: Xcode → Preferences → Accounts → Manage Certificates → "+"

19. Keychain and Certificates



> ⚠️ If the private key is missing from Keychain, code signing will fail even if the certificate exists.

---

20. Development ID

- **Apple ID** associated with Developer Program membership
- **Team ID** — 10-char alphanumeric identifier (e.g. `A1B2C3D4E5`)
- Used in provisioning profiles and entitlements

---

21. Code Signing Identity

The specific certificate used: `Apple Development: John Doe (A1B2C3D4E5)`

Set in: **Build Settings** → **Signing** → **Code Signing Identity**

---

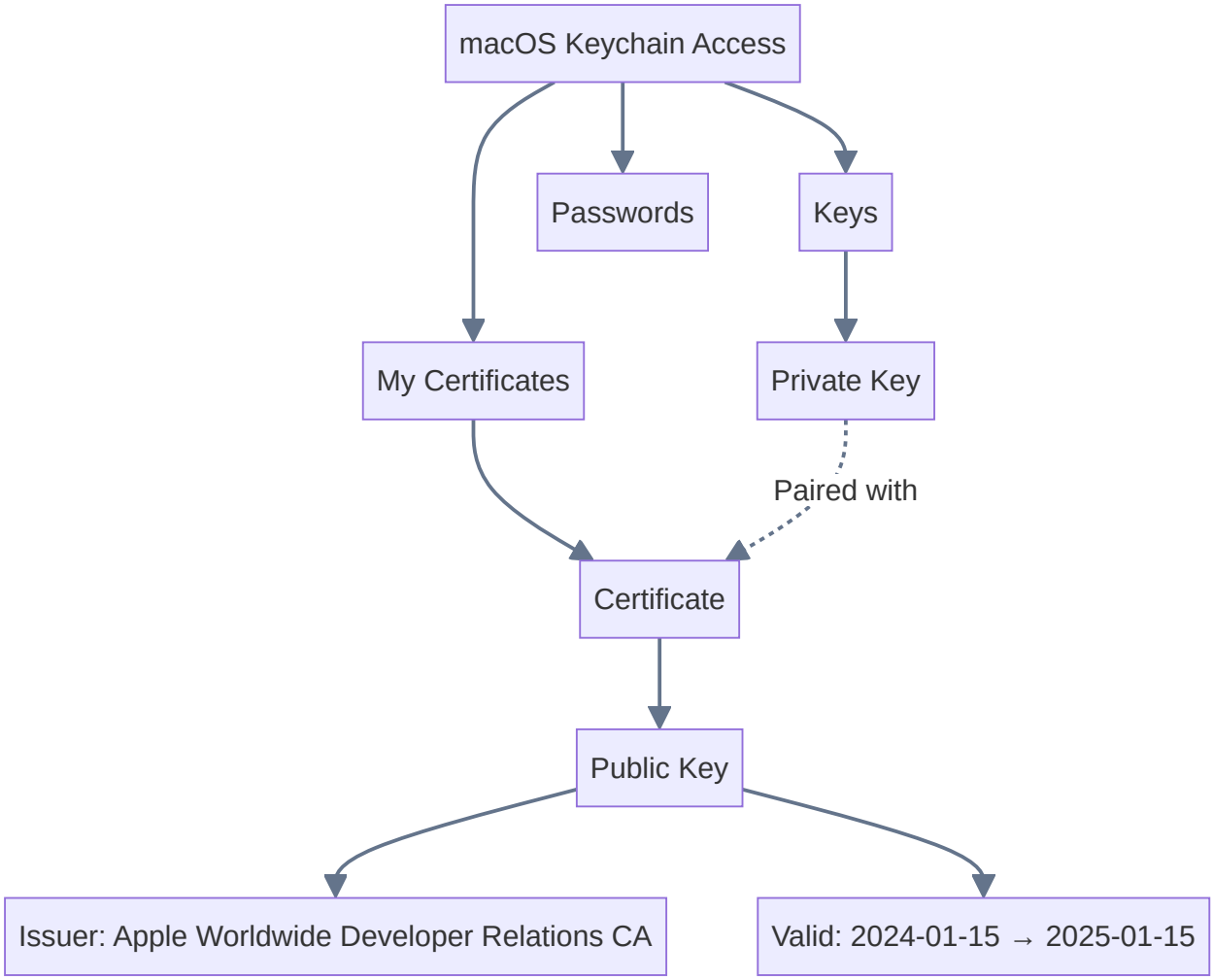
22. Bundle ID

Unique iOS app identifier: `com.companyname.appname`

- Must match across Xcode project, provisioning profile, and App Store Connect
- Wildcard: `com.companyname.*` (development only)

---

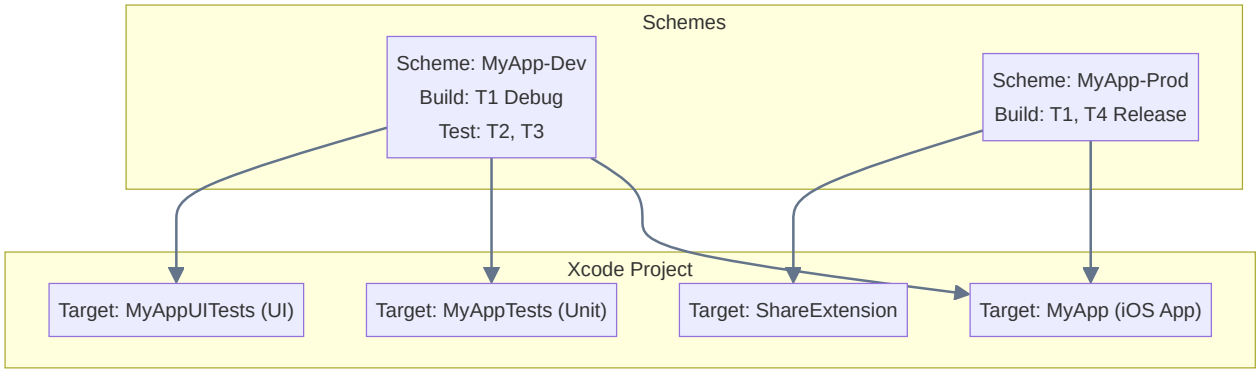
23. Keychain Deep Dive



Both certificate + private key = valid Code Signing Identity

---

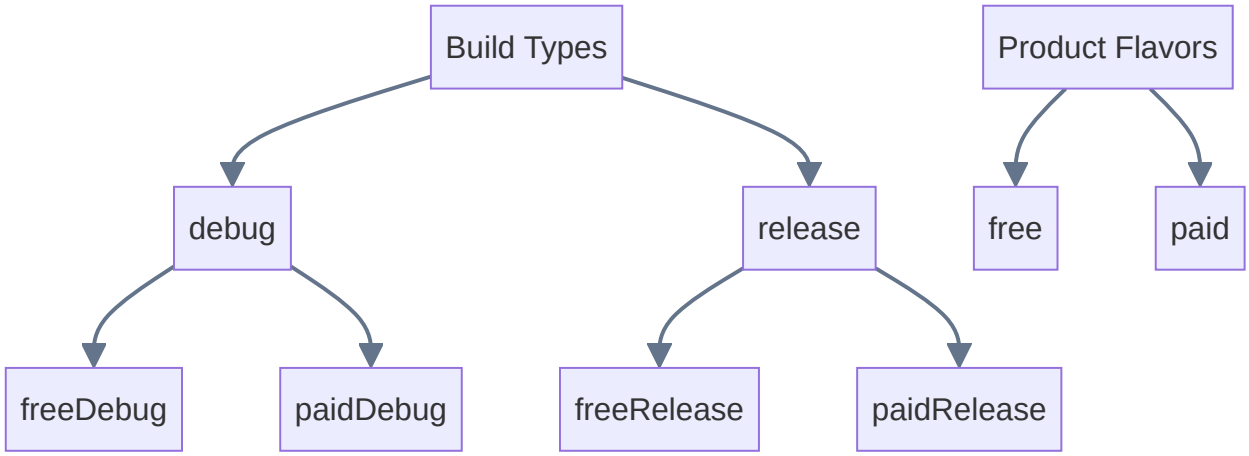
24. Schemes vs Targets in iOS



Concept	Definition
Target	**What** to build — a product (app, framework, extension, test suite)
Scheme	**How** to build — which targets, order, build config, actions

25. Build Variants (Android)

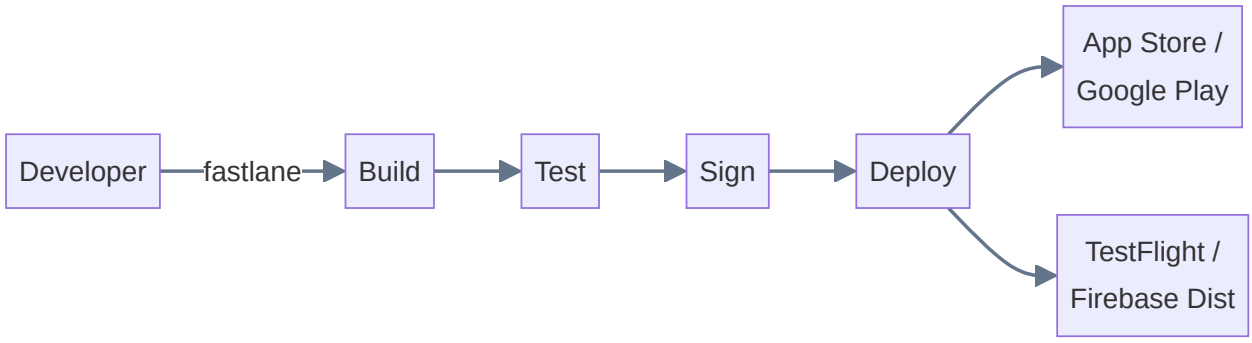
Build Variant = Build Type × Product Flavor




```
android {
    buildTypes {
        release { minifyEnabled true }
    }
    productFlavors {
        free { applicationIdSuffix ".free" }
        paid { applicationIdSuffix ".paid" }
    }
}
```

26 & 30. Fastlane

Open-source automation tool for iOS/Android build, test, sign, and deploy.

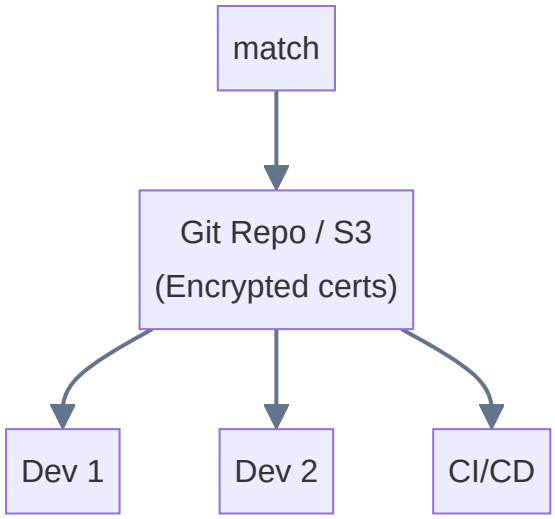


Tool	Purpose
`gym`	Build iOS apps
`deliver`	Upload to App Store
`pilot`	Upload to TestFlight
`supply`	Upload to Google Play
`match`	Sync code signing across team
`scan`	Run tests

Fastfile Example:

```
lane :beta do
  increment_build_number
  build_app(scheme: "MyApp")
  upload_to_testflight
  slack(message: "New beta uploaded!")
end
```

match — Team Code Signing



---

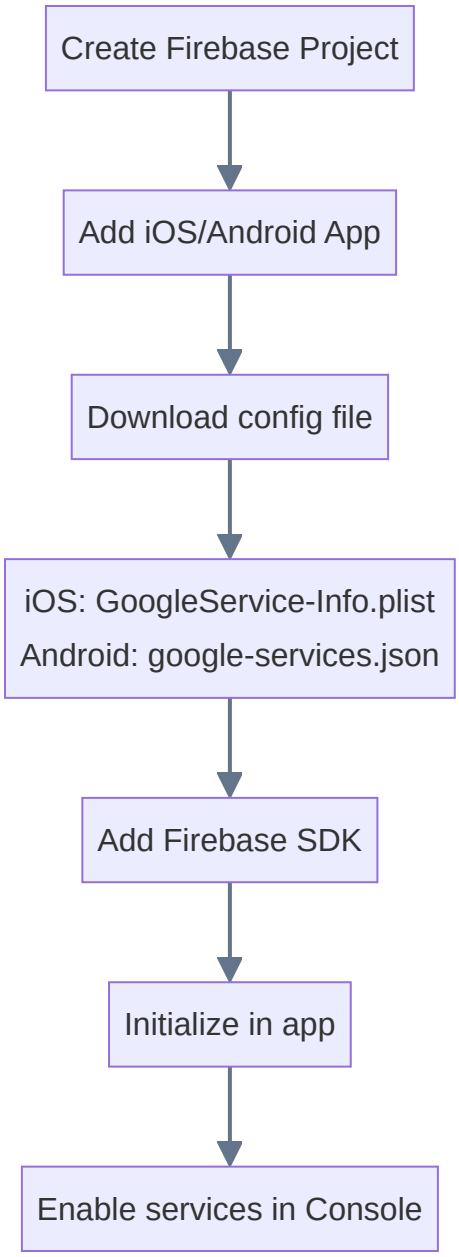
27. Crashlytics — Integration

1. Add Firebase SDK + Crashlytics dependency
2. **iOS:** Upload dSYM files for symbolication
3. **Android:** Upload ProGuard/R8 mapping files
4. Force a test crash to verify setup

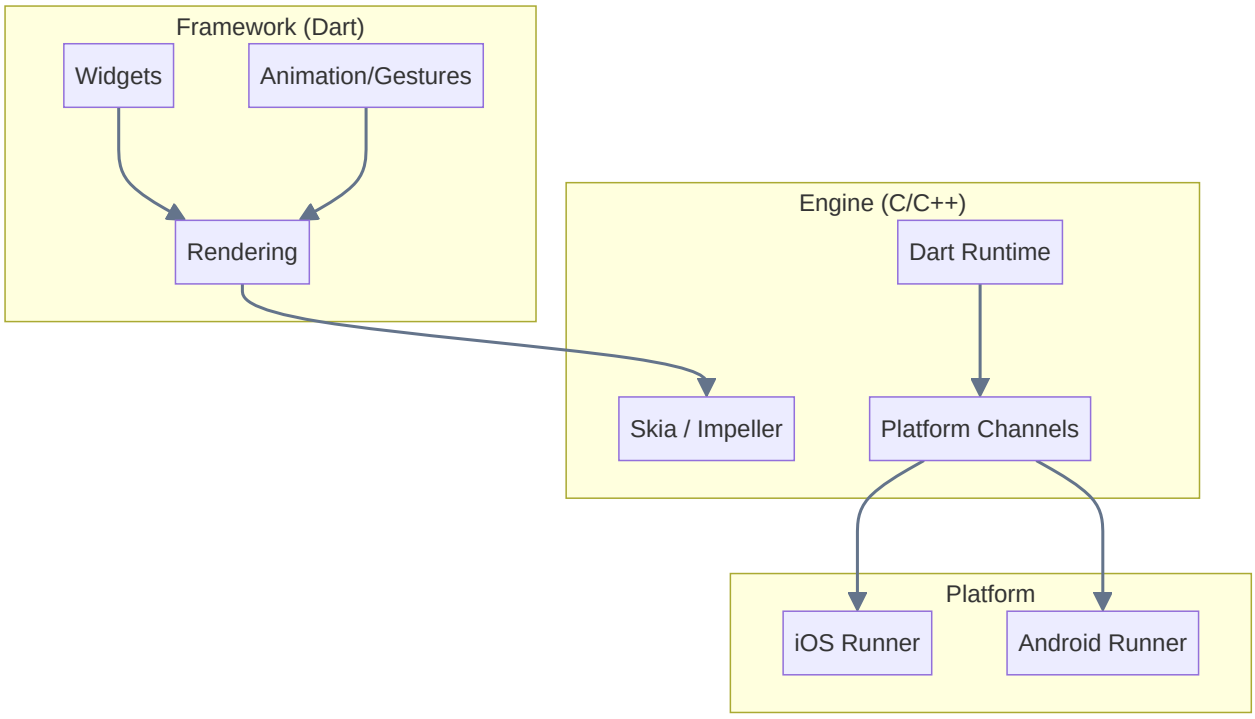
dSYM: Maps memory addresses → function names + line numbers. Without it, crash logs show only addresses.

---

28. Firebase Setup Flow

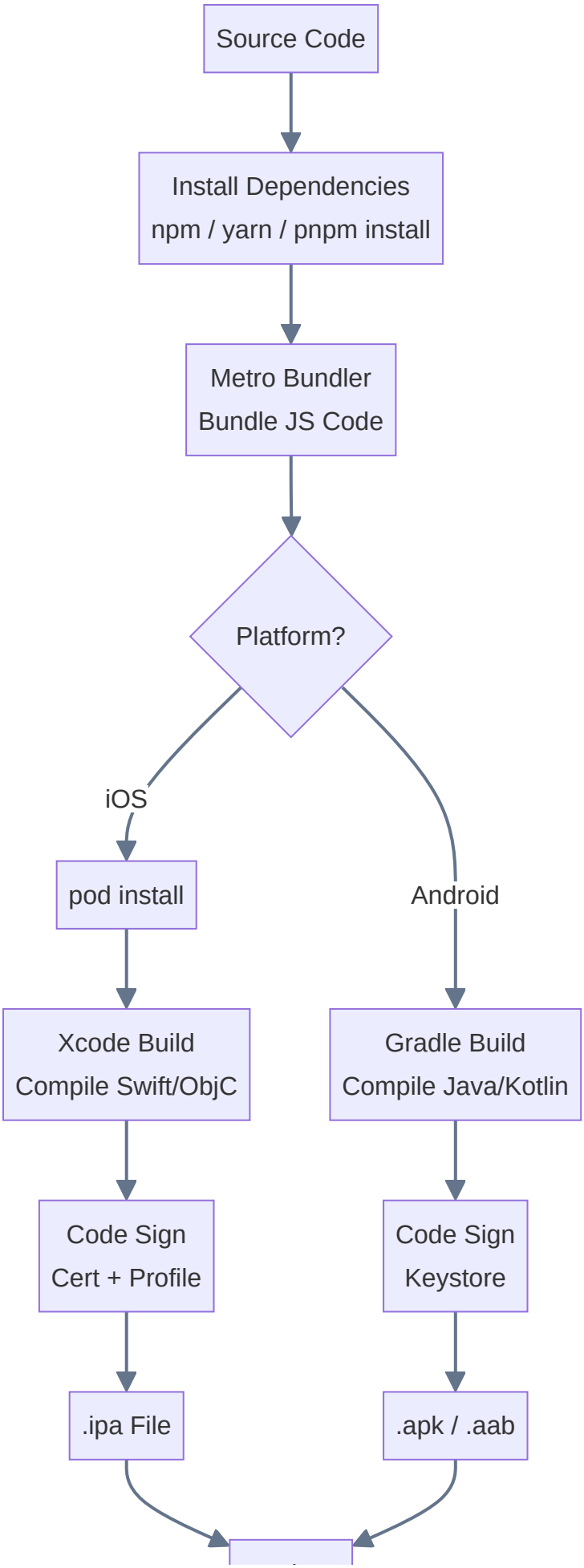


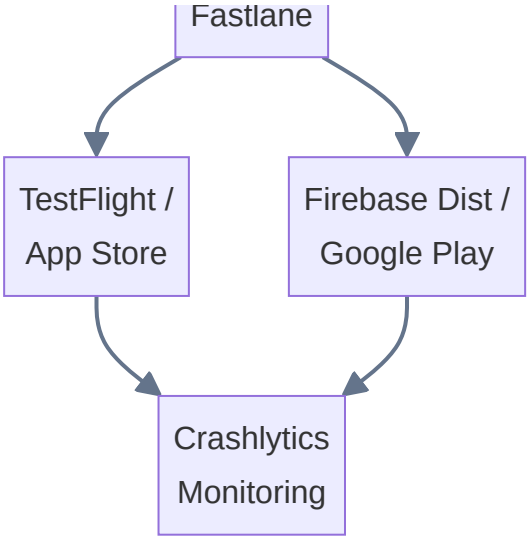
29. Flutter Architecture



Build Mode	Use Case	Compilation
Debug	Development	JIT (slow, hot reload)
Profile	Performance testing	Near-release
Release	Production	AOT (optimized)

Part 3: Complete Build & Deploy Flow


Source Code
Install Dependencies
npm / yarn / pnpm install
Metro Bundler
Bundle JS Code
Platform?
iOS
Android
pod install
Xcode Build
Compile Swift/ObjC
Gradle Build
Compile Java/Kotlin
Code Sign
Cert + Profile
Code Sign
Keystore
.ipa File
.apk / .aab



Mobile App Development & DevOps Reference Guide — Version 1.0 — March 2026